

10. Colecciones y diccionarios

10.1 Colecciones: la clase `ArrayList`

Una colección en Java es una estructura de datos que permite almacenar muchos valores del mismo tipo; por tanto, conceptualmente es prácticamente igual que un *array*. Según el uso y según si se permiten o no repeticiones, Java dispone de un amplio catálogo de colecciones: `ArrayList` (lista), `ArrayBlockingQueue` (cola), `HashSet` (conjunto), `Stack` (pila), etc. En este manual estudiaremos la colección `ArrayList`.

Un `ArrayList` es una estructura en forma de lista que permite almacenar elementos del mismo tipo (pueden ser incluso objetos); su tamaño va cambiando a medida que se añaden o se eliminan esos elementos.

Nos podemos imaginar un `ArrayList` como un conjunto de celdas o cajoncitos donde se guardan los valores, exactamente igual que un *array* convencional. En la práctica será más fácil trabajar con un `ArrayList`.

En capítulos anteriores hemos podido comprobar la utilidad del *array*; es un recurso imprescindible que cualquier programador debe manejar con soltura. No obstante, el *array* presenta algunos inconvenientes. Uno de ellos es la necesidad de conocer el tamaño exacto en el momento de su creación. Una colección, sin embargo, se crea sin que se tenga que especificar el tamaño; posteriormente se van añadiendo y quitando elementos a medida que se necesitan.

Trabajando con *arrays* es frecuente cometer errores al utilizar los índices; por ejemplo al intentar guardar un elemento en una posición que no existe (índice fuera de rango). Aunque las colecciones permiten el uso de índices, no es necesario indicarlos siempre. Por ejemplo, en una colección del tipo `ArrayList`, cuando hay que añadir el elemento "Amapola", se puede hacer simplemente `flores.add("Amapola")`. Al no especificar índice, el elemento "Amapola" se añadiría justo al final de `flores` independientemente del tamaño y del número de elementos que se hayan introducido ya.

La clase `ArrayList` es muy similar a la clase `Vector`. Ésta última está obsoleta y, por tanto, no se recomienda su uso.

Principales métodos de `ArrayList`

Las operaciones más comunes que se pueden realizar con un objeto de la clase `ArrayList` son las siguientes:

add(elemento)

Añade un elemento al final de la lista.

add(indice, elemento)

Inserta un elemento en una posición determinada, desplazando el resto de elementos hacia la derecha.

clear()

Elimina todos los elementos pero no borra la lista.

contains(elemento)

Devuelve `true` si la lista contiene el elemento que se especifica y `false` en caso contrario.

get(indice)

Devuelve el elemento de la posición que se indica entre paréntesis.

indexOf(elemento)

Devuelve la posición de la primera ocurrencia del elemento que se indica entre paréntesis.

isEmpty()

Devuelve `true` si la lista está vacía y `false` en caso de tener algún elemento.

remove(indice)

Elimina el elemento que se encuentra en una posición determinada.

remove(elemento)

Elimina la primera ocurrencia de un elemento.

removeIf(filtro)

Elimina los elementos que cumplen una determinada condición.

set(indice, elemento)

Machaca el elemento que se encuentra en una determinada posición con el elemento que se pasa como parámetro.

size()

Devuelve el tamaño (número de elementos) de la lista.

toArray()

Devuelve un *array* con todos y cada uno de los elementos que contiene la lista.

Puedes consultar todos los métodos disponibles en la [documentación oficial de la clase ArrayList](http://docs.oracle.com/javase/7/docs/api/java/util/ArrayList.html)¹.

¹ <http://docs.oracle.com/javase/7/docs/api/java/util/ArrayList.html>

Definición de un `ArrayList` e inserción, borrado y modificación de sus elementos

A continuación se muestra un ejemplo en el que se puede ver cómo se declara un `ArrayList` y cómo se insertan y se extraen elementos.

```
import java.util.ArrayList;

/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList01 {
    public static void main(String[] args) {

        ArrayList<String> a = new ArrayList<String>();

        System.out.println("Nº de elementos: " + a.size());

        a.add("rojo");
        a.add("verde");
        a.add("azul");

        System.out.println("Nº de elementos: " + a.size());

        a.add("blanco");

        System.out.println("Nº de elementos: " + a.size());

        System.out.println("El elemento que hay en la posición 0 es " + a.get(0));
        System.out.println("El elemento que hay en la posición 3 es " + a.get(3));
    }
}
```

Observa que al crear un objeto de la clase `ArrayList` hay que indicar el tipo de dato que se almacenará en las celdas de esa lista. Para ello se utilizan los caracteres `<` y `>`. No hay que olvidar los paréntesis del final.



Es necesario importar la clase `ArrayList` para poder crear objetos de esta clase, para ello debe aparecer al principio del programa la línea `import java.util.ArrayList;`. Algunos IDEs (por ej. Netbeans) insertan esta línea de código de forma automática.

En el siguiente ejemplo se muestra un `ArrayList` de números enteros.

```
import java.util.ArrayList;

/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList011 {
    public static void main(String[] args) {

        ArrayList<Integer> a = new ArrayList<Integer>();

        a.add(18);
        a.add(22);
        a.add(-30);

        System.out.println("Nº de elementos: " + a.size());

        System.out.println("El elemento que hay en la posición 1 es " + a.get(1));
    }
}
```

Se define la estructura de la siguiente manera:

```
ArrayList<Integer> a = new ArrayList<Integer>();
```

Fíjate que no se utiliza el tipo simple `int` sino el wrapper `Integer`. Recuerda que los wrapper son clases que engloban a los tipos simples y les añaden nuevas funcionalidades (p. ej. permiten tratar a las variables numéricas como objetos). El wrapper de `int` es `Integer`, el de `float` es `Float`, el de `double` es `Double`, el de `long` es `Long`, el de `boolean` es `Boolean` y el de `char` es `Character`.

En el siguiente ejemplo podemos ver cómo extraer todos los elementos de una lista a la manera tradicional, con un bucle `for`.

```
import java.util.ArrayList;

/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList02 {
    public static void main(String[] args) {

        ArrayList<String> a = new ArrayList<String>();
```

```
a.add("rojo");
a.add("verde");
a.add("azul");
a.add("blanco");
a.add("amarillo");

System.out.println("Contenido de la lista: ");

for(int i=0; i < a.size(); i++) {
    System.out.println(a.get(i));
}
}
```

Si estás acostumbrado al `for` clásico, habrás visto que es muy sencillo recorrer todos los elementos del `ArrayList`. No obstante, al trabajar con colecciones es recomendable usar el `for` al estilo `foreach` como se muestra en el siguiente ejemplo. Como puedes ver, la sintaxis es más corta y no se necesita crear un índice para recorrer la estructura.

```
import java.util.ArrayList;

/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList03 {
    public static void main(String[] args) {

        ArrayList<String> a = new ArrayList<String>();

        a.add("rojo");
        a.add("verde");
        a.add("azul");
        a.add("blanco");
        a.add("amarillo");

        System.out.println("Contenido de la lista: ");

        for(String color: a) {
            System.out.println(color);
        }
    }
}
```

Fíjate en estas líneas:

```
for(String color: a) {  
    System.out.println(color);  
}
```

El significado de este código sería el siguiente:

```
for(String color: a)
```

→ Saca uno a uno todos los elementos de `a` y ve metiéndolos en una variable de nombre `color`.

```
System.out.println(color);
```

→ Muestra por pantalla el contenido de la variable `color`.

Veamos ahora en otro ejemplo cómo eliminar elementos de un `ArrayList`. Se utiliza el método `remove()` y se puede pasar como parámetro el índice del elemento que se quiere eliminar, o bien el valor del elemento. O sea, `a.remove(2)` elimina el elemento que se encuentra en la posición 2 de `a` mientras que `a.remove("blanco")` elimina el valor "blanco".

Es importante destacar que el `ArrayList` se reestructura de forma automática después del borrado de cualquiera de sus elementos.

```
import java.util.ArrayList;  
  
/**  
 * Ejemplo de uso de la clase ArrayList  
 *  
 * @author Luis José Sánchez  
 */  
public class EjemploArrayList04 {  
    public static void main(String[] args) {  
  
        ArrayList<String> a = new ArrayList<String>();  
  
        a.add("rojo");  
        a.add("verde");  
        a.add("azul");  
        a.add("blanco");  
        a.add("amarillo");  
        a.add("blanco");  
  
        System.out.println("Contenido de la lista: ");
```

```

    for(String color: a) {
        System.out.println(color);
    }

    if (a.contains("blanco")) {
        System.out.println("El blanco está en la lista de colores");
    }

    a.remove("blanco");

    System.out.println("Contenido de la lista después de quitar la " +
        "primera ocurrencia del color blanco: ");

    for(String color: a) {
        System.out.println(color);
    }

    a.remove(2);
    System.out.println("Contenido de la lista después de quitar el " +
        "elemento de la posición 2: ");

    for(String color: a) {
        System.out.println(color);
    }
}

```

A continuación se muestra un ejemplo en el que se “machaca” una posición del `ArrayList`. Al hacer `a.set(2, "turquesa")`, se borra lo que hubiera en la posición 2 y se coloca el valor "turquesa". Sería equivalente a hacer `a[2] = "turquesa"` en caso de que `a` fuese un `array` tradicional en lugar de un `ArrayList`.

```

import java.util.ArrayList;

/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList05 {
    public static void main(String[] args) {

        ArrayList<String> a = new ArrayList<String>();

        a.add("rojo");
    }
}

```

```

        a.add("verde");
        a.add("azul");
        a.add("blanco");
        a.add("amarillo");

        System.out.println("Contenido del vector: ");

        System.out.println(a);

        a.set(2, "turquesa");

        System.out.println("Contenido del vector después de machacar la posición 2: ");

        System.out.println(a);
    }
}

```

El método `add` permite añadir elementos a un `ArrayList` como ya hemos visto. Por ejemplo, `a.add("amarillo")` añade el elemento "amarillo" al final de `a`. Este método se puede utilizar también con un índice de la forma `a.add(1, "turquesa")`. En este caso, lo que se hace es insertar en la posición indicada. Lo mejor de todo es que el `ArrayList` se reestructura de forma automática desplazando el resto de elementos.

```

import java.util.ArrayList;

/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList06 {
    public static void main(String[] args) {

        ArrayList<String> a = new ArrayList<String>();

        a.add("rojo");
        a.add("verde");
        a.add("azul");
        a.add("blanco");
        a.add("amarillo");

        System.out.println("Contenido de la lista: ");

        System.out.println(a);

        a.add(1, "turquesa");
    }
}

```



```
        System.out.println("Contenido de la lista después de insertar en la posición 1: ");

        System.out.println(a);
    }
}
```

Mediante `removeIf` se puede hacer un borrado selectivo según si los elementos de una lista cumplen una determinada condición. Por ejemplo, podemos eliminar todas las palabras que contienen la letra "a".

```
import java.util.ArrayList;

/**
 * Ejemplo de uso de la clase ArrayList.
 * Borrado de elementos con removeIf().
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList066 {
    public static void main(String[] args) {

        ArrayList<String> a = new ArrayList<String>();

        a.add("rojo");
        a.add("verde");
        a.add("azul");
        a.add("blanco");
        a.add("amarillo");

        System.out.println("Contenido de la lista: ");
        System.out.println(a);

        a.removeIf(palabra -> palabra.contains("a"));

        System.out.println("Contenido de la lista después de borrar las palabras que contienen la \
letra \"a\": ");
        System.out.println(a);

    }
}
```

En el siguiente ejemplo se hace otro borrado selectivo. Se eliminan todos los números menores que 10.

```
import java.util.ArrayList;

/**
 * Ejemplo de uso de la clase ArrayList.
 * Borrado de elementos con removeIf().
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList067 {
    public static void main(String[] args) {

        ArrayList<Integer> a = new ArrayList<Integer>();

        a.add(20);
        a.add(7);
        a.add(14);
        a.add(32);
        a.add(3);

        System.out.println("Contenido de la lista: ");
        System.out.println(a);

        a.removeIf(numero -> numero < 10);

        System.out.println("Contenido de la lista después de borrar los menores de 10: ");
        System.out.println(a);

    }
}
```

ArrayList de objetos

Una colección `ArrayList` puede contener objetos que son instancias de clases definidas por el programador. Esto es muy útil sobre todo en aplicaciones de gestión para guardar datos de alumnos, productos, libros, etc.

En el siguiente ejemplo, definimos una lista de gatos. En cada celda de la lista se almacenará un objeto de la clase `Gato`.

```
import java.util.ArrayList;

/**
 * Uso de un ArrayList de objetos
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList07 {
    public static void main(String[] args) {

        ArrayList<Gato> g = new ArrayList<Gato>();

        g.add(new Gato("Garfield", "naranja", "mestizo"));
        g.add(new Gato("Pepe", "gris", "angora"));
        g.add(new Gato("Mauri", "blanco", "manx"));
        g.add(new Gato("Ulises", "marrón", "persa"));

        System.out.println("\nDatos de los gatos:\n");

        for (Gato gatoAux: g) {
            System.out.println(gatoAux+"\n");
        }
    }
}
```

En el siguiente apartado se muestra la definición de la clase `Gato`.

Ordenación de un `ArrayList`

Los elementos de una lista se pueden ordenar con el método `sort`. El formato es el siguiente:

```
Collections.sort(lista);
```

Observa que `sort` es un método de clase que está definido en `Collections`. Para poder utilizar este método es necesario incluir la línea

```
import java.util.Collections;
```

al principio del programa. A continuación se muestra un ejemplo del uso de `sort`.

```
import java.util.ArrayList;
import java.util.Collections;

/**
 * Ordenación de un ArrayList
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList071 {
    public static void main(String[] args) {

        ArrayList<Integer> a = new ArrayList<Integer>();

        a.add(67);
        a.add(78);
        a.add(10);
        a.add(4);

        System.out.println("\nNúmeros en el orden original:");
        for (int numero: a) {
            System.out.println(numero);
        }

        Collections.sort(a);

        System.out.println("\nNúmeros ordenados:");
        for (int numero: a) {
            System.out.println(numero);
        }
    }
}
```

También es posible ordenar una lista de objetos. En este caso es necesario indicar el criterio de ordenación en la definición de la clase. En el programa principal, se utiliza el método `sort` igual que si se tratase de una lista de números o de palabras como se muestra a continuación.

```
import java.util.ArrayList;
import java.util.Collections;

/**
 * Ordenación de un ArrayList de objetos
 *
 * @author Luis José Sánchez
 */
public class EjemploArrayList08 {
```

```

public static void main(String[] args) {

    ArrayList<Gato> g = new ArrayList<Gato>();

    g.add(new Gato("Garfield", "naranja", "mestizo"));
    g.add(new Gato("Pepe", "gris", "angora"));
    g.add(new Gato("Mauri", "blanco", "manx"));
    g.add(new Gato("Ulises", "marrón", "persa"));
    g.add(new Gato("Adán", "negro", "angora"));

    Collections.sort(g);

    System.out.println("\nDatos de los gatos ordenados por nombre:");

    for (Gato gatoAux: g) {
        System.out.println(gatoAux+"\n");
    }
}

```

Ahora bien, en la definición de la clase `Gato` hay que indicar de alguna manera cómo se debe realizar la ordenación, ya que Java no sabe de antemano si los gatos se ordenan según el color, el nombre, el peso, etc.

Lo primero que hay que hacer es indicar que los objetos de la clase `Gato` se pueden comparar unos con otros. Para ello, cambiamos la siguiente línea:

```
public class Gato
```

por esta otra:

```
public class Gato implements Comparable<Gato>
```

Lo siguiente y no menos importante es definir el método `compareTo`. Este método debe devolver un 0 si los elementos que se comparan son iguales, un número negativo si el primer elemento que se compara es menor que el segundo y un número positivo en caso contrario. Afortunadamente, las clases `String`, `Integer`, `Double`, etc. ya tienen implementado su propio método `compareTo` así que tenemos hecho lo más difícil. Lo único que deberemos escribir en nuestro código es un `compareTo` con los atributos que queremos comparar.

En el caso que nos ocupa, si queremos ordenar los gatos por nombre, tendremos que implementar el `compareTo` de la clase `Gato` de tal forma que nos devuelva el resultado del `compareTo` de los nombres de los gatos que estamos comparando, de la siguiente manera:

```
public int compareTo(Gato g) {  
    return (this.nombre).compareTo(g.getNombre());  
}
```

Si en lugar de ordenar por nombre, quisiéramos ordenar por raza, el método `compareTo` de la clase `Gato` sería el siguiente:

```
public int compareTo(Gato g) {  
    return (this.raza).compareTo(g.getRaza());  
}
```

A continuación se muestra la definición completa de la clase `Gato`.

```
/**  
 * Definición de la clase Gato  
 *  
 * @author  
 */  
public class Gato implements Comparable<Gato> {  
    private String nombre;  
    private String color;  
    private String raza;  
  
    public Gato(String nombre, String color, String raza) {  
        this.nombre = nombre;  
        this.color = color;  
        this.raza = raza;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public String getRaza() {  
        return raza;  
    }  
  
    public String toString() {  
        return "Nombre: " + this.nombre + "\nColor: " + this.color + "\nRaza: " + this.raza;  
    }  
  
    public int compareTo(Gato g) {  
        return (this.nombre).compareTo(g.getNombre());  
    }  
}
```

```
public boolean equals(Gato g) {  
    return (this.nombre).equals(g.getNombre());  
}  
}
```

10.2 Diccionarios: la clase `HashMap`

Imagina un diccionario inglés-español. Queremos saber qué significa la palabra “*stiff*”. Sabemos que en el diccionario hay muchas entradas y en cada entrada tenemos una palabra en inglés y su correspondiente traducción al español. Buscando por la “s” encontramos que “*stiff*” significa “agujetas”.

Un diccionario en Java funciona exactamente igual. Contiene una serie de elementos que son las entradas que a su vez están formadas por un par (clave, valor). La clave (*key*) permite acceder al valor. No puede haber claves duplicadas. En el ejemplo anterior, la clave sería “*stiff*” y el valor “agujetas”.

Java dispone de varios tipos de diccionarios: `HashMap`, `EnumMap`, `Hashtable`, `IdentityHashMap`, `LinkedHashMap`, etc. Nosotros estudiaremos el diccionario `HashMap`.

Principales métodos de `HashMap`

Algunos de los métodos más importantes de la clase `HashMap` son:

`get(clave)`

Obtiene el valor correspondiente a una clave. Devuelve `null` si no existe esa clave en el diccionario.

`put(clave, valor)`

Añade un par (clave, valor) al diccionario. Si ya había un valor para esa clave, se machaca.

`keySet()`

Devuelve un conjunto (set) con todas las claves.

`values()`

Devuelve una colección con todos los valores (los valores pueden estar duplicados a diferencia de las claves).

`entrySet()`

Devuelve una colección con todos los pares (clave, valor).

`containsKey(clave)`

Devuelve `true` si el diccionario contiene la clave indicada y `false` en caso contrario.

`getKey()`

Devuelve la clave de la entrada. Se aplica a una sola entrada del diccionario (no al diccionario completo), es decir a una pareja (clave, valor).

Por ejemplo:

```
for (Map.Entry pareja: m.entrySet()) {  
    System.out.println(pareja.getKey());  
}
```

getValue()

Devuelve el contenido de la entrada. Se aplica a una entrada del diccionario (no al diccionario completo), es decir a una pareja (clave, valor).

Por ejemplo:

```
for (Map.Entry pareja: m.entrySet()) {  
    System.out.println(pareja.getValue());  
}
```

Definición de un `HashMap` e inserción, borrado y modificación de entradas

Al declarar un diccionario hay que indicar los tipos tanto de la clave como del valor. En el siguiente ejemplo definiremos el diccionario `m` que tendrá como clave un número entero y una cadena de caracteres como valor. Este diccionario se declara de esta forma:

```
HashMap<Integer, String> m = new HashMap<Integer, String>();
```

No hay que olvidar importar la clase al principio del programa:

```
import java.util.HashMap;
```

Para insertar una entrada en el diccionario se utiliza el método `put` indicando siempre la clave y el valor. Veamos un ejemplo completo.

```
import java.util.HashMap;  
  
/**  
 * Ejemplo de uso de la clase HashMap  
 *  
 * @author Luis José Sánchez  
 */  
public class EjemploHashMap01 {  
    public static void main(String[] args) {
```



```

HashMap<Integer, String> m = new HashMap<Integer, String>();

m.put(924, "Amalia Núñez");
m.put(921, "Cindy Nero");
m.put(700, "César Vázquez");
m.put(219, "Víctor Tilla");
m.put(537, "Alan Brito");
m.put(605, "Esteban Quito ");

System.out.println("Los elementos de m son: \n" + m);
}
}

```

Para extraer valores se utiliza el método `get`. Se proporciona una clave y el diccionario nos devuelve el valor, igual que un diccionario de verdad. Si no existe ninguna entrada con la clave que se indica, se devuelve `null`.

```

import java.util.HashMap;

/**
 * Ejemplo de uso de la clase HashMap
 *
 * @author Luis José Sánchez
 */
public class EjemploHashMap011 {
    public static void main(String[] args) {

        HashMap<Integer, String> m = new HashMap<Integer, String>();

        m.put(924, "Amalia Núñez");
        m.put(921, "Cindy Nero");
        m.put(700, "César Vázquez");
        m.put(219, "Víctor Tilla");
        m.put(537, "Alan Brito");
        m.put(605, "Esteban Quito ");

        System.out.println(m.get(921));
        System.out.println(m.get(605));
        System.out.println(m.get(888));
    }
}

```

¿Y si queremos extraer todas las entradas? Tenemos varias opciones. Podemos usar el método `print` directamente sobre el diccionario de la forma `System.out.print(diccionario)` como vimos en un ejemplo anterior; de esta manera se muestran por pantalla todas las entradas encerradas entre llaves. También podemos convertir el diccionario en

un `entrySet` (conjunto de entradas) y mostrarlo con `print`; de esta forma se obtiene una salida por pantalla muy parecida a la primera (en lugar de llaves se muestran corchetes). Otra opción es utilizar un `for` para recorrer una a una todas las entradas. En este último caso hay que convertir el diccionario en un `entrySet` ya que no se pueden sacar las entradas directamente del diccionario. Estas dos últimas opciones se ilustran en el siguiente ejemplo.

```
import java.util.HashMap;
import java.util.Map;

/**
 * Ejemplo de uso de la clase HashMap
 *
 * @author Luis José Sánchez
 */
public class EjemploHashMap02 {
    public static void main(String[] args) {

        HashMap<Integer, String> m = new HashMap<Integer, String>();

        m.put(924, "Amalia Núñez");
        m.put(921, "Cindy Nero");
        m.put(700, "César Vázquez");
        m.put(219, "Víctor Tilla");
        m.put(537, "Alan Brito");
        m.put(605, "Esteban Quito ");

        System.out.println("Todas las entradas del diccionario extraídas con entrySet:");
        System.out.println(m.entrySet());

        System.out.println("\nEntradas del diccionario extraídas una a una:");
        for (Map.Entry pareja: m.entrySet()) {
            System.out.println(pareja);
        }
    }
}
```

A continuación se muestra el uso de los métodos `getKey` y `getValue` que extraen la clave y el valor de una entrada respectivamente.

```
import java.util.HashMap;
import java.util.Map;

/**
 * Ejemplo de uso de la clase HashMap
 *
 * @author Luis José Sánchez
 */
public class EjemploHashMap03 {
    public static void main(String[] args) {

        HashMap<Integer, String> m = new HashMap<Integer, String>();

        m.put(924, "Amalia Núñez");
        m.put(921, "Cindy Nero");
        m.put(700, "César Vázquez");
        m.put(219, "Víctor Tilla");
        m.put(537, "Alan Brito");
        m.put(605, "Esteban Quito ");

        System.out.println("Código\tNombre\n-----\t-----");

        for (Map.Entry pareja: m.entrySet()) {
            System.out.print(pareja.getKey() + "\t");
            System.out.println(pareja.getValue());
        }
    }
}
```

En el último programa de ejemplo hacemos uso del método `containsKey` que nos servirá para saber si existe o no una determinada clave en un diccionario y del método `get` que, como ya hemos visto, sirve para extraer un valor a partir de su clave.

```
import java.util.HashMap;

/**
 * Ejemplo de uso de la clase HashMap
 *
 * @author Luis José Sánchez
 */
public class EjemploHashMap04 {
    public static void main(String[] args) {

        HashMap<Integer, String> m = new HashMap<Integer, String>();

        m.put(924, "Amalia Núñez");
```

```
m.put(921, "Cindy Nero");
m.put(700, "César Vázquez");
m.put(219, "Víctor Tilla");
m.put(537, "Alan Brito");
m.put(605, "Esteban Quito ");

System.out.print("Por favor, introduzca un código: ");
int codigoIntroducido = Integer.parseInt(System.console().readLine());

if (m.containsKey(codigoIntroducido)) {
    System.out.print("El código " + codigoIntroducido + " corresponde a ");
    System.out.println(m.get(codigoIntroducido));
} else {
    System.out.print("El código introducido no existe.");
}
}
```

10.3 Ejercicios



Ejercicio 1

Crea un `ArrayList` con los nombres de 6 compañeros de clase. A continuación, muestra esos nombres por pantalla. Utiliza para ello un bucle `for` que recorra todo el `ArrayList` sin usar ningún índice.



Ejercicio 2

Realiza un programa que introduzca valores aleatorios (entre 0 y 100) en un `ArrayList` y que luego calcule la suma, la media, el máximo y el mínimo de esos números. El tamaño de la lista también será aleatorio y podrá oscilar entre 10 y 20 elementos ambos inclusive.



Ejercicio 3

Escribe un programa que ordene 10 números enteros introducidos por teclado y almacenados en un objeto de la clase `ArrayList`.



Ejercicio 4

Realiza un programa equivalente al anterior pero en esta ocasión, el programa debe ordenar palabras en lugar de números.



Ejercicio 5

Realiza de nuevo el ejercicio de la colección de discos pero utilizando esta vez una lista para almacenar la información sobre los discos en lugar de un *array* convencional. Comprobarás que el código se simplifica notablemente ¿Cuánto ocupa el programa original hecho con un *array*? ¿Cuánto ocupa este nuevo programa hecho con una lista?



Ejercicio 6

Implementa el control de acceso al área restringida de un programa. Se debe pedir un nombre de usuario y una contraseña. Si el usuario introduce los datos correctamente, el programa dirá “Ha accedido al área restringida”. El usuario tendrá un máximo de 3 oportunidades. Si se agotan las oportunidades el programa dirá “Lo siento, no tiene acceso al área restringida”. Los nombres de usuario con sus correspondientes contraseñas deben estar almacenados en una estructura de la clase `HashMap`.



Ejercicio 7

La máquina *Eurocoin* genera una moneda de curso legal cada vez que se pulsa un botón siguiendo la siguiente pauta: o bien coincide el valor con la moneda anteriormente generada - 1 céntimo, 2 céntimos, 5 céntimos, 10 céntimos, 25 céntimos, 50 céntimos, 1 euro o 2 euros - o bien coincide la posición - cara o cruz. Simula, mediante un programa, la generación de 6 monedas aleatorias siguiendo la pauta correcta. Cada moneda generada debe ser una instancia de la clase `Moneda` y la secuencia se debe ir almacenando en una lista.

Ejemplo:

```
2 céntimos - cara
2 céntimos - cruz
50 céntimos - cruz
1 euro - cruz
1 euro - cara
10 céntimos - cara
```



Ejercicio 8

Realiza un programa que escoja al azar 10 cartas de la baraja española (10 objetos de la clase `Carta`). Emplea un objeto de la clase `ArrayList` para almacenarlas y asegúrate de que no se repite ninguna.



Ejercicio 9

Modifica el programa anterior de tal forma que las cartas se muestren ordenadas. Primero se ordenarán por palo: bastos, copas, espadas, oros. Cuando coincida el palo, se ordenará por número: as, 2, 3, 4, 5, 6, 7, sota, caballo, rey.



Ejercicio 10

Crea un mini-diccionario español-inglés que contenga, al menos, 20 palabras (con su correspondiente traducción). Utiliza un objeto de la clase `HashMap` para almacenar las parejas de palabras. El programa pedirá una palabra en español y dará la correspondiente traducción en inglés.



Ejercicio 11

Realiza un programa que escoja al azar 5 palabras en español del mini-diccionario del ejercicio anterior. El programa irá pidiendo que el usuario teclee la traducción al inglés de cada una de las palabras y comprobará si son correctas. Al final, el programa deberá mostrar cuántas respuestas son válidas y cuántas erróneas.



Ejercicio 12

Escribe un programa que genere una secuencia de 5 cartas de la baraja española y que sume los puntos según el juego de la brisca. El valor de las cartas se debe guardar en una estructura `HashMap` que debe contener parejas (figura, valor), por ejemplo ("caballo", 3). La secuencia de cartas debe ser una estructura de la clase `ArrayList` que contiene objetos de la clase `Carta`. El valor de las cartas es el siguiente: as → 11, tres → 10, sota → 2, caballo → 3, rey → 4; el resto de cartas no vale nada.

Ejemplo:

```
as de oros
cinco de bastos
caballo de espadas
sota de copas
tres de oros
Tienes 26 puntos
```



Ejercicio 13

Modifica el programa **Gestisimal** realizado anteriormente añadiendo las siguientes mejoras:

- Utiliza una lista en lugar de un *array* para el almacenamiento de los datos.
- Comprueba la existencia del código en el alta, la baja y la modificación de artículos para evitar errores.
- Cambia la opción "Salida de stock" por "Venta". Esta nueva opción permitirá hacer una venta de varios artículos y emitir la factura correspondiente. Se debe preguntar por los códigos y las cantidades de cada artículo que se quiere comprar. Aplica un 21% de IVA.



Ejercicio 14

Un supermercado de productos ecológicos nos ha pedido hacer un programa para vender su mercancía. En esta primera versión del programa se tendrán en cuenta los productos que se indican en la tabla junto con su precio. Los productos se venden en bote, brick, etc. Cuando se realiza la compra, hay que indicar el producto y el número de unidades que se compran, por ejemplo **“guisantes”** si se quiere comprar un bote de guisantes y la cantidad, por ejemplo **“3”** si se quieren comprar 3 botes. La compra se termina con la palabra **“fin”**. Suponemos que el usuario no va a intentar comprar un producto que no existe. Utiliza un diccionario para almacenar los nombres y precios de los productos y una o varias listas para almacenar la compra que realiza el usuario.

A continuación se muestra una tabla con los productos disponibles y sus respectivos precios:

avena	garbanzos	tomate	jengibre	quinoa	guisantes
2,21	2,39	1,59	3,13	4,50	1,60

Ejemplo:

```
Producto: tomate
Cantidad: 1
Producto: quinoa
Cantidad: 2
Producto: avena
Cantidad: 1
Producto: tomate
Cantidad: 2
Producto: fin
```

```
Producto Precio Cantidad Subtotal
-----
tomate    1,59      1      1,59
quinoa    4,50      2      9,00
avena     2,21      1      2,21
tomate    1,59      2      3,18
-----
TOTAL: 15,98
```




Ejercicio 15

Realiza una nueva versión del ejercicio anterior con las siguientes mejoras: Si algún producto se repite en diferentes líneas, se deben agrupar en una sola. Por ejemplo, si se pide primero 1 bote de tomate y luego 3 botes de tomate, en el extracto se debe mostrar que se han pedido 4 botes de tomate. Después de teclear “fin”, el programa pide un código de descuento. Si el usuario introduce el código “ECODTO”, se aplica un 10% de descuento en la compra.

Ejemplo:

```
Producto: tomate
Cantidad: 1
Producto: quinoa
Cantidad: 2
Producto: avena
Cantidad: 1
Producto: quinoa
Cantidad: 2
Producto: tomate
Cantidad: 2
Producto: fin
Introduzca código de descuento (INTRO si no tiene ninguno): ECODTO
```

Producto	Precio	Cantidad	Subtotal
tomate	1,59	3	4,77
quinoa	4,50	4	18,00
avena	2,21	1	2,21

Descuento: 2,50

TOTAL: 22,48



Ejercicio 16

Realiza un programa que sepa decir la capital de un país (en caso de conocer la respuesta) y que, además, sea capaz de aprender nuevas capitales. En principio, el programa solo conoce las capitales de España, Portugal y Francia. Estos datos deberán estar almacenados en un diccionario. Los datos sobre capitales que vaya aprendiendo el programa se deben almacenar en el mismo diccionario. El usuario sale del programa escribiendo la palabra “salir”.

Ejemplo:

```
Escribe el nombre de un país y te diré su capital: España
La capital de España es Madrid
Escribe el nombre de un país y te diré su capital: Alemania
No conozco la respuesta ¿cuál es la capital de Alemania?: Berlín
Gracias por enseñarme nuevas capitales
Escribe el nombre de un país y te diré su capital: Portugal
La capital de Portugal es Lisboa
Escribe el nombre de un país y te diré su capital: Alemania
La capital de Alemania es Berlín
Escribe el nombre de un país y te diré su capital: Francia
La capital de Francia es París
Escribe el nombre de un país y te diré su capital: salir
```



Ejercicio 17

Una empresa de venta por internet de productos electrónicos nos ha encargado implementar un carrito de la compra. Crea la clase **Carrito**. Al carrito se le pueden ir agregando elementos que se guardarán en una lista, por tanto, deberás crear la clase **Elemento**. Cada elemento del carrito deberá contener el nombre del **producto**, su **precio** y la **cantidad** (número de unidades de dicho producto). A continuación se muestra tanto el contenido del programa principal como la salida que debe mostrar el programa. Los métodos a implementar se pueden deducir del main.

Contenido del main:

```
Carrito miCarrito = new Carrito();
miCarrito.agrega(new Elemento("Tarjeta SD 64Gb", 19.95, 2));
miCarrito.agrega(new Elemento("Canon EOS 2000D", 449, 1));
System.out.println(miCarrito);
System.out.print("Hay " + miCarrito.numeroDeElementos());
System.out.println(" productos en la cesta.");
System.out.println("El total asciende a "
    + String.format("%.2f", miCarrito.importeTotal()) + " euros");
```

```
System.out.println("\nContinúa la compra...\n");
miCarrito.agrega(new Elemento("Samsung Galaxy Tab", 199, 3));
miCarrito.agrega(new Elemento("Tarjeta SD 64Gb", 19.95, 1));
System.out.println(miCarrito);
System.out.print("Ahora hay " + miCarrito.numeroDeElementos());
System.out.println(" productos en la cesta.");
System.out.println("El total asciende a "
    + String.format("%.2f", miCarrito.importeTotal()) + " euros");
```

Salida:

```
Contenido del carrito
=====
Tarjeta SD 64Gb PVP: 19,95 Unidades: 2 Subtotal: 39,90
Canon EOS 2000D PVP: 449,00 Unidades: 1 Subtotal: 449,00
```

```
Hay 2 productos en la cesta.
El total asciende a 488,90 euros
```

```
Continúa la compra...
```

```
Contenido del carrito
=====
Tarjeta SD 64Gb PVP: 19,95 Unidades: 2 Subtotal: 39,90
Canon EOS 2000D PVP: 449,00 Unidades: 1 Subtotal: 449,00
Samsung Galaxy Tab PVP: 199,00 Unidades: 3 Subtotal: 597,00
Tarjeta SD 64Gb PVP: 19,95 Unidades: 1 Subtotal: 19,95
```

```
Ahora hay 4 productos en la cesta.
El total asciende a 1105,85 euros
```



Ejercicio 18

Mejora el programa anterior (en otro proyecto diferente) de tal forma que al intentar agregar un elemento al carrito, se compruebe si ya existe el producto y, en tal caso, se incremente el número de unidades sin añadir un nuevo elemento. Observa que en el programa anterior, se repetía el producto “Tarjeta SD 64Gb” dos veces en el carrito. En esta nueva versión ya no sucede esto, si no que se incrementa el número de unidades del producto que se agrega.

El contenido del main es idéntico al ejercicio anterior.

Salida:

Contenido del carrito

=====

Tarjeta SD 64Gb PVP: 19,95 Unidades: 2 Subtotal: 39,90

Canon EOS 2000D PVP: 449,00 Unidades: 1 Subtotal: 449,00

Hay 2 productos en la cesta.

El total asciende a 488,90 euros

Continúa la compra...

Contenido del carrito

=====

Tarjeta SD 64Gb PVP: 19,95 Unidades: 3 Subtotal: 59,85

Canon EOS 2000D PVP: 449,00 Unidades: 1 Subtotal: 449,00

Samsung Galaxy Tab PVP: 199,00 Unidades: 3 Subtotal: 597,00

Ahora hay 3 productos en la cesta.

El total asciende a 1105,85 euros



Ejercicio 19

Realiza un buscador de sinónimos. Utiliza el diccionario español-inglés que se proporciona a continuación. El programa preguntará una palabra y dará una lista de sinónimos (palabras que tienen el mismo significado). Por ejemplo, si se introduce la palabra “caliente”, el programa dará como resultado: ardiente, candente, abrasador. ¿Cómo sabe el programa cuáles son los sinónimos de “caliente”? Muy fácil, en el diccionario debe existir la entrada (“caliente”, “hot”), por tanto solo tendrá que buscar las palabras en español que también signifiquen “hot”; esta información estará en las entradas (“ardiente”, “hot”) y (“abrasador”, “hot”). Cuando una palabra existe en el diccionario pero no tiene sinónimos, debe mostrar el mensaje “No conozco sinónimos de esa palabra”. Si una palabra no está en el diccionario se mostrará el mensaje “No conozco esa palabra”. El usuario sale del programa escribiendo la palabra “salir”.

Español	caliente	rojo	ardiente	verde	agujetas	abrasador	hierro	grande
Inglés	hot	red	hot	green	stiff	hot	iron	big

Ejemplo:

Introduzca una palabra y le daré los sinónimos: caliente

Sinónimos de caliente: ardiente, abrasador

Introduzca una palabra y le daré los sinónimos: rojo

No conozco sinónimos de esa palabra

Introduzca una palabra y le daré los sinónimos: blanco

No conozco esa palabra
Introduzca una palabra y le daré los sinónimos: grande
No conozco sinónimos de esa palabra
Introduzca una palabra y le daré los sinónimos: salir



Ejercicio 20

Amplía el programa anterior de tal forma que sea capaz de aprender palabras y sinónimos. Cuando una palabra no tiene sinónimos, es decir, cuando aparece la palabra en español con su traducción y esa traducción no la tiene ninguna otra palabra española, se le preguntará al usuario si quiere añadir uno (un sinónimo) y, en caso afirmativo, se pedirá la palabra y se añadirá al diccionario. Se puede dar la circunstancia de que el usuario introduzca una palabra en español que no está en el diccionario; en tal caso, se mostrará el consiguiente mensaje y se dará la posibilidad al usuario de añadir la entrada correspondiente en el diccionario pidiendo, claro está, la palabra en inglés.

Ejemplo:

Introduzca una palabra y le daré los sinónimos: caliente
Sinónimos de caliente: ardiente, abrasador
Introduzca una palabra y le daré los sinónimos: rojo
No conozco sinónimos de esa palabra ¿quiere añadir alguno? (s/n): s
Introduzca un sinónimo de rojo: colorado
Gracias por enseñarme nuevos sinónimos.
Introduzca una palabra y le daré los sinónimos: blanco
No conozco esa palabra ¿quiere añadirla al diccionario? (s/n): s
Introduzca la traducción de blanco en inglés: white
Introduzca una palabra y le daré los sinónimos: rojo
Sinónimos de rojo: colorado
Introduzca una palabra y le daré los sinónimos: blanco
No conozco sinónimos de esa palabra ¿quiere añadir alguno? (s/n): s
Introduzca un sinónimo de blanco: albino
Gracias por enseñarme nuevos sinónimos.
Introduzca una palabra y le daré los sinónimos: blanco
Sinónimos de blanco: albino
Introduzca una palabra y le daré los sinónimos: salir



Ejercicio 21

La asociación “Amigos de los anfibios” nos ha encargado una aplicación educativa sobre estos animalitos. Crea un programa que pida al usuario el tipo de anfibio y que, a continuación, nos muestre su hábitat y su alimentación. Si el tipo de anfibio introducido no existe, se debe mostrar el mensaje “Ese tipo de anfibio no existe”.

Ejemplo 1:

Introduzca el tipo de anfibio: salamandra

Hábitat: Ecosistemas húmedos.

Alimentación: Pequeños crustáceos e insectos.

Ejemplo 2:

Introduzca el tipo de anfibio: gato

Ese tipo de anfibio no existe.

La información se debe guardar en dos diccionarios (dos HashMap). Uno de ellos tendrá parejas clave-valor del tipo **(tipo de anfibio, hábitat)** y otro **(tipo de anfibio, alimentación)**. A continuación se muestra la información en una tabla:

				
Tipo de anfibio	rana	salamandra	sapo	tritón
Hábitat	En los trópicos y cerca de las zonas húmedas y acuáticas	Ecosistemas húmedos.	En cualquier sitio salvo el desierto y la Antártida.	América y África.
Alimentación	Larvas e insectos.	Pequeños crustáceos e insectos.	Insectos, lombrices y pequeños roedores.	Insectos

**Ejercicio 22**

Amplía el [ejercicio 13 del capítulo anterior](#) que implementaba cuentas corrientes de un banco de tal forma que cada cuenta lleve un registro de todos los movimientos realizados: ingresos, cargos y transferencias (tanto enviadas como recibidas).

Contenido del `main`

```
CuentaCorriente cuenta1 = new CuentaCorriente();
CuentaCorriente cuenta2 = new CuentaCorriente(1500);
CuentaCorriente cuenta3 = new CuentaCorriente(6000);
cuenta1.ingreso(2000);
cuenta1.cargo(600);
cuenta3.ingreso(75);
cuenta1.cargo(55);
cuenta2.transferencia(cuenta1, 100);
cuenta1.transferencia(cuenta3, 250);
cuenta3.transferencia(cuenta1, 22);
cuenta1.movimientos();
```

Salida

Movimientos de la cuenta 1654432813

Ingreso de 2000 € Saldo: 2000,00 €

Cargo de 600 € Saldo: 1400,00 €

Cargo de 55 € Saldo: 1345,00 €

Transf. recibida de 100 € de la cuenta 1654432813 Saldo 1445,00 €

Transf. emitida de 250 € a la cuenta 6546817008 Saldo 1195,00 €

Transf. recibida de 22 € de la cuenta 1654432813 Saldo 1217,00 €



Ejercicio 23

En ajedrez, el valor de las piezas se mide en peones. Una **dama** vale 9 peones, una **torre** 5 peones, un **alfil** 3, un **caballo** 2 y un **peón** vale, lógicamente, 1 peón. Realiza un programa que genere al azar las capturas que ha hecho un jugador durante una partida. El número de capturas será un valor aleatorio entre 0 y 15. Hay que tener en cuenta que cada jugador tiene la posibilidad de capturar algunas de las siguientes piezas (no más): 1 dama, 2 torres, 2 alfiles, 2 caballos y 8 peones. Al final debe aparecer la puntuación total.

Ejemplo:

Fichas capturadas por el jugador:

Alfil (3 peones)

Caballo (2 peones)

Peón (1 peones)

Torre (5 peones)

Peón (1 peones)

Puntos totales: 12 peones.